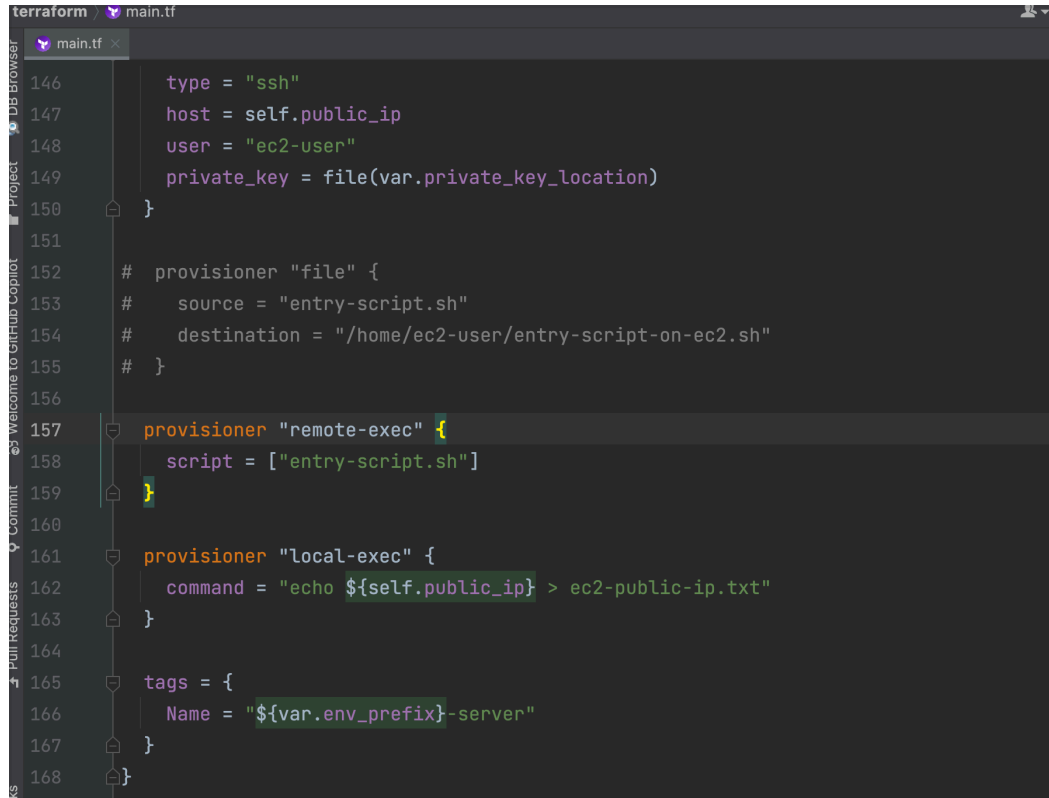


Module 12 - chapter 16

Modules in Terraform - Part 1

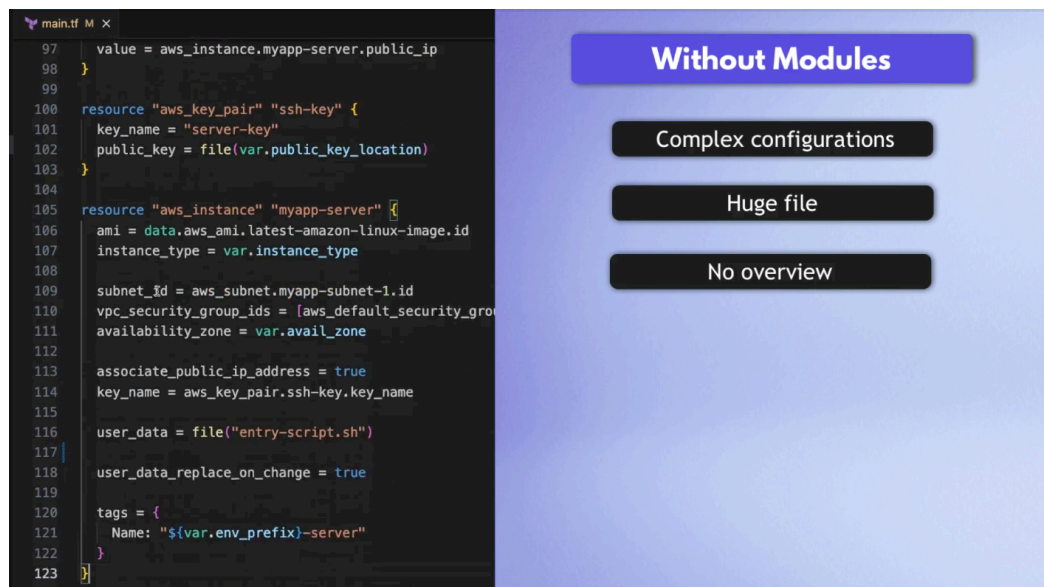
So far we have build a simple infrastructure with TF and the last edition was to create an EC2 instance, and at this point we have over 100 lines of code in our TF file.

Containing variables, resources, data, outputs

A screenshot of a code editor window titled 'main.tf'. The editor shows Terraform code for an EC2 instance. The code is organized into blocks with line numbers on the left. The code includes an SSH block (lines 146-150), a file provisioner (lines 152-155), a remote-exec provisioner (lines 157-159), a local-exec provisioner (lines 161-163), and a tags block (lines 165-167). The code uses variables like 'self.public_ip', 'var.private_key_location', and 'var.env_prefix'.

```
146     type = "ssh"
147     host = self.public_ip
148     user = "ec2-user"
149     private_key = file(var.private_key_location)
150 }
151
152 # provisioner "file" {
153 #   source = "entry-script.sh"
154 #   destination = "/home/ec2-user/entry-script-on-ec2.sh"
155 # }
156
157 provisioner "remote-exec" {
158     script = ["entry-script.sh"]
159 }
160
161 provisioner "local-exec" {
162     command = "echo ${self.public_ip} > ec2-public-ip.txt"
163 }
164
165 tags = {
166     Name = "${var.env_prefix}-server"
167 }
168 }
```

So navigating the code (looking for something) becomes difficult the more lines we have.



In terraform we have the concept of MODULES

To move from a monolithic TF to a modular TF -> we break up parts of our configuration into logical groups

We package the groups together in folders and these folders represent **Modules**

Modules

= container for multiple resources, used together

Why Modules?

Organize and group
configurations

Encapsulate into
distinct **logical components**

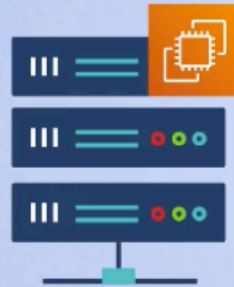
Re-use

For example, every resource we need to create an EC2 instance will be package in one module

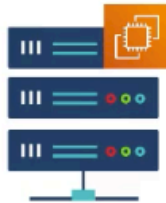
- Data "aws_ami"
- Resource "aws_key_pair"
- resource "aws_default_security_group"
- resource "aws_instance"

Modules

Module for EC2
Instance



"webserver" module



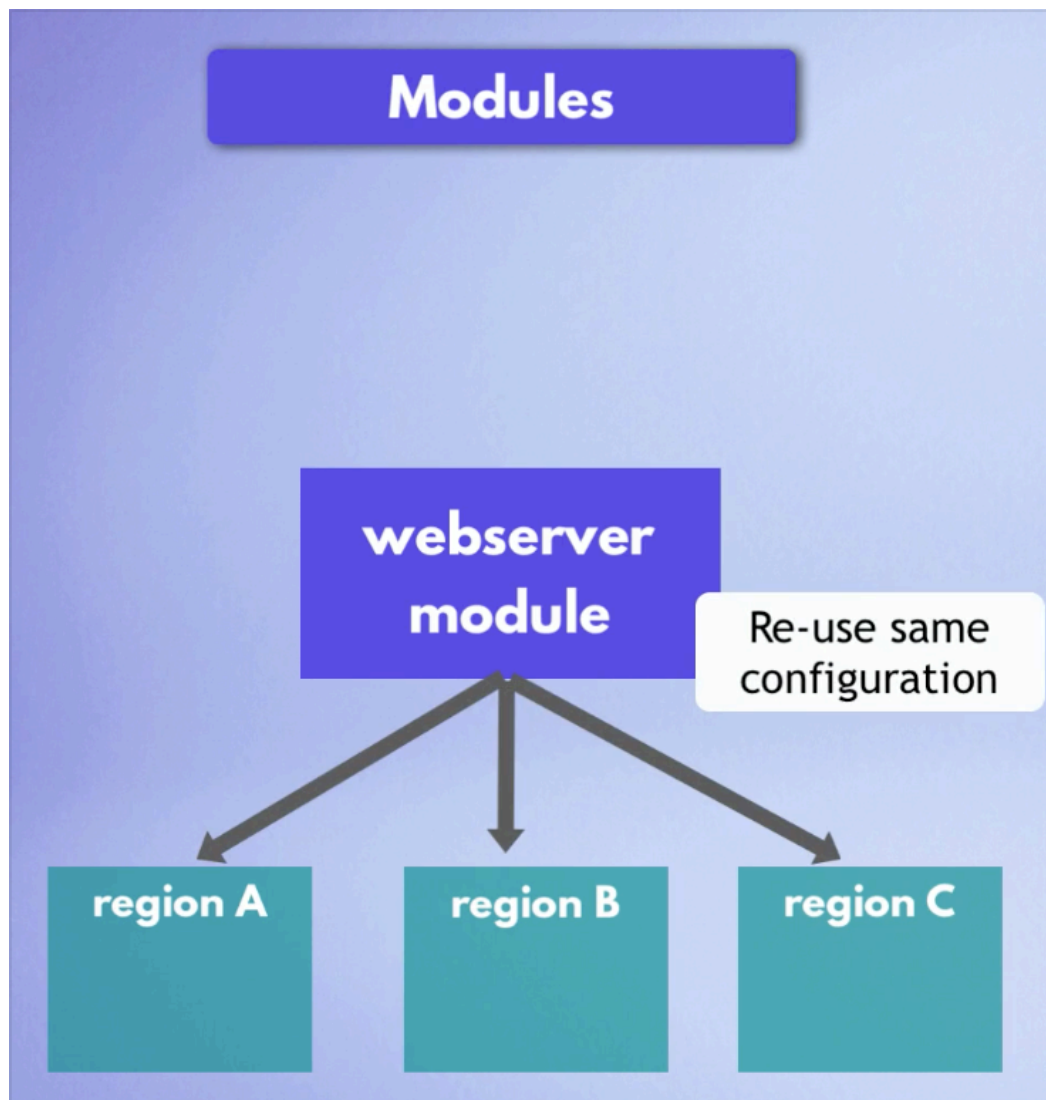
And we can all it -> web server module

Modules

**webserver
module**

Now we have a module that we can **reuse**

For example: if we deploy infra with multiple EC2 instances in different regions and different AZ, we can reuse that same web server module



And whenever we define a module we can pass in any parameters that we want

Modules

customize the configuration with variables

input variables



**webserver
module**

And we can also access the output of the module

Modules

customize the configuration with variables

input variables



**webserver
module**



output values

expose created resources or specific attributes

Analogy with programming:

Module -> function

Like Function Definition



input variables = like function arguments

output values = like function return values

A module must be a proper abstraction of the resources -> so having only one component in a module makes no sense like only 'resource "aws_instance"'

```
main.tf M X
102 public_key = file(var.public_key_location)
103 }
104
105 resource "aws_instance" "myapp-server" {
106   ami = data.aws_ami.latest-amazon-linux-image.id
107   instance_type = var.instance_type
108
109   subnet_id = aws_subnet.myapp-subnet-1.id
110   vpc_security_group_ids = [aws_default_security_group.default-sg.id]
111   availability_zone = var.avail_zone
112
113   associate_public_ip_address = true
114   key_name = aws_key_pair.ssh-key.key_name
115
116   user_data = file("entry-script.sh")
117   user_data_replace_on_change = true
118
119   tags = {
120     Name = "${var.env_prefix}-server"
121   }
122 }
123 }
```

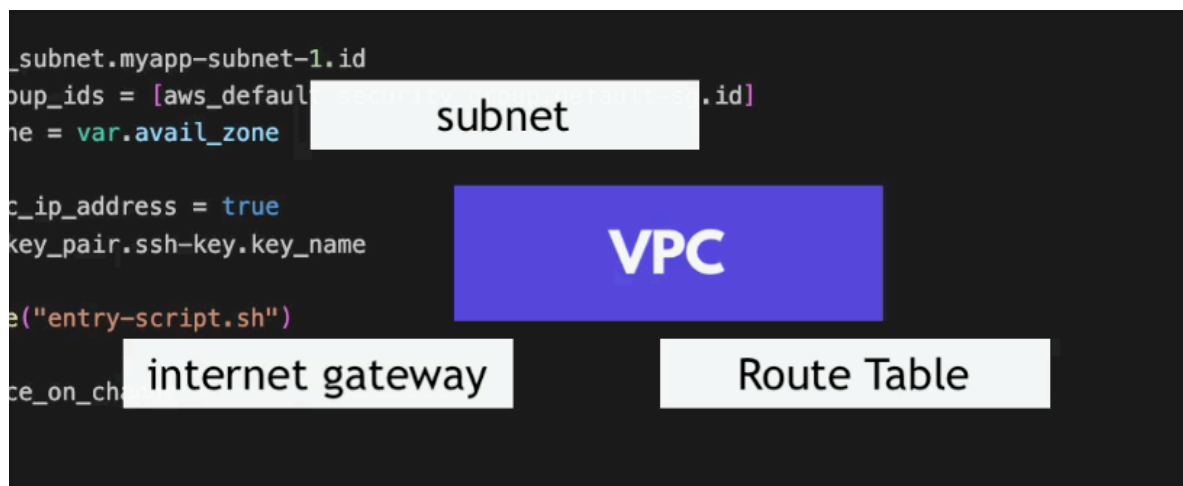
Proper/logical abstraction

A module it should contain at least 2 resources that logically go together.

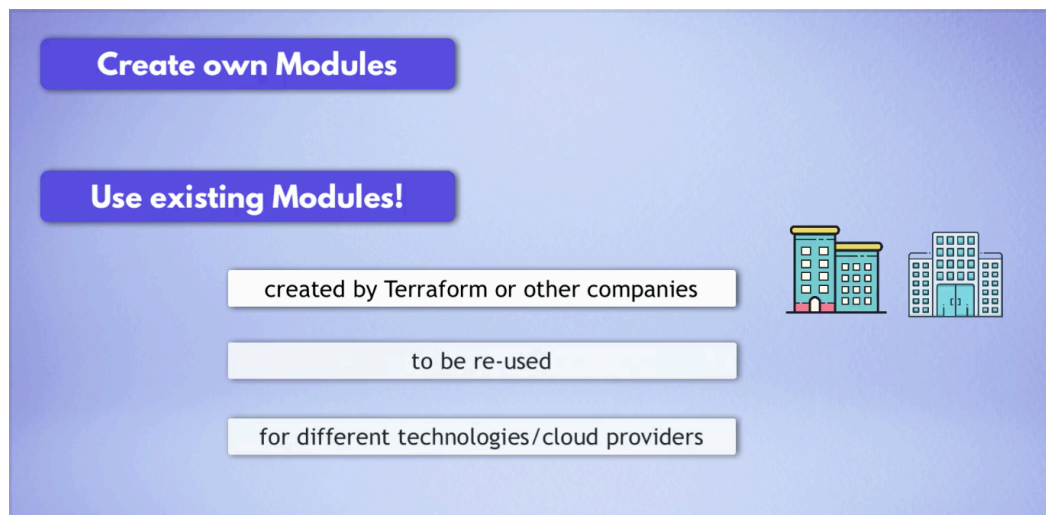
For network configuration -> If we create a VPC we need:

- Subnet
- IGW - internet gateway
- Route Table

So the above can go in one module called "vpc"

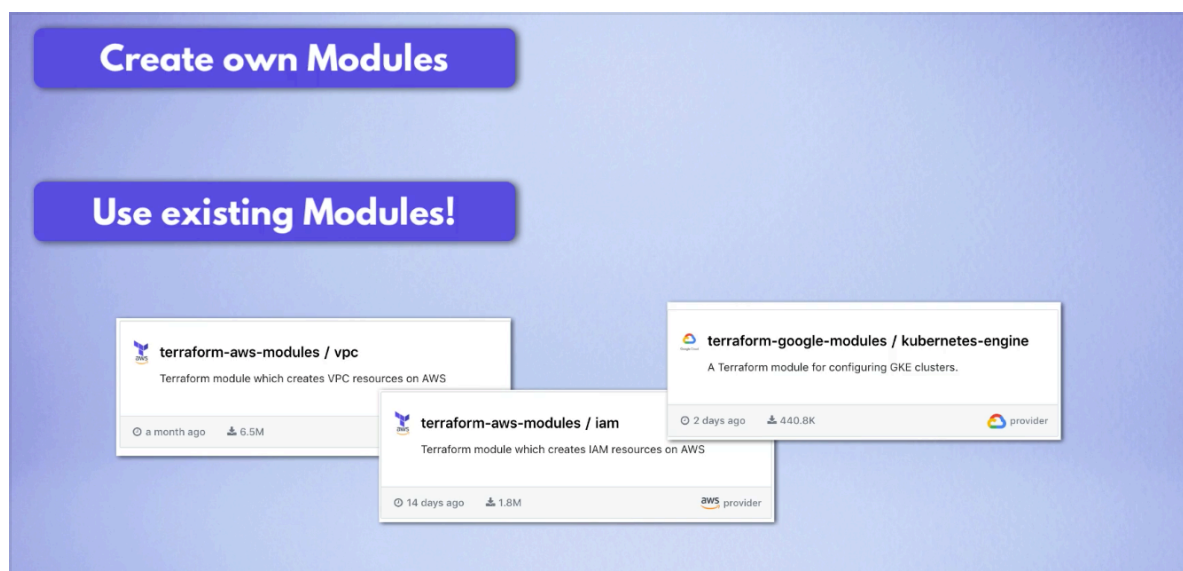


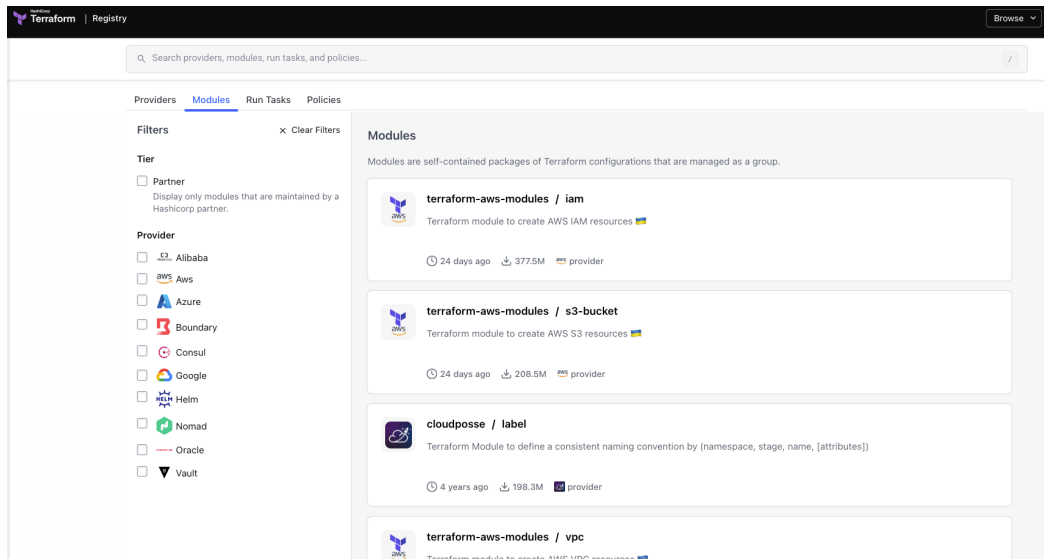
We can create our own Modules, BUT for common use cases there are already Modules created by TF or other companies.



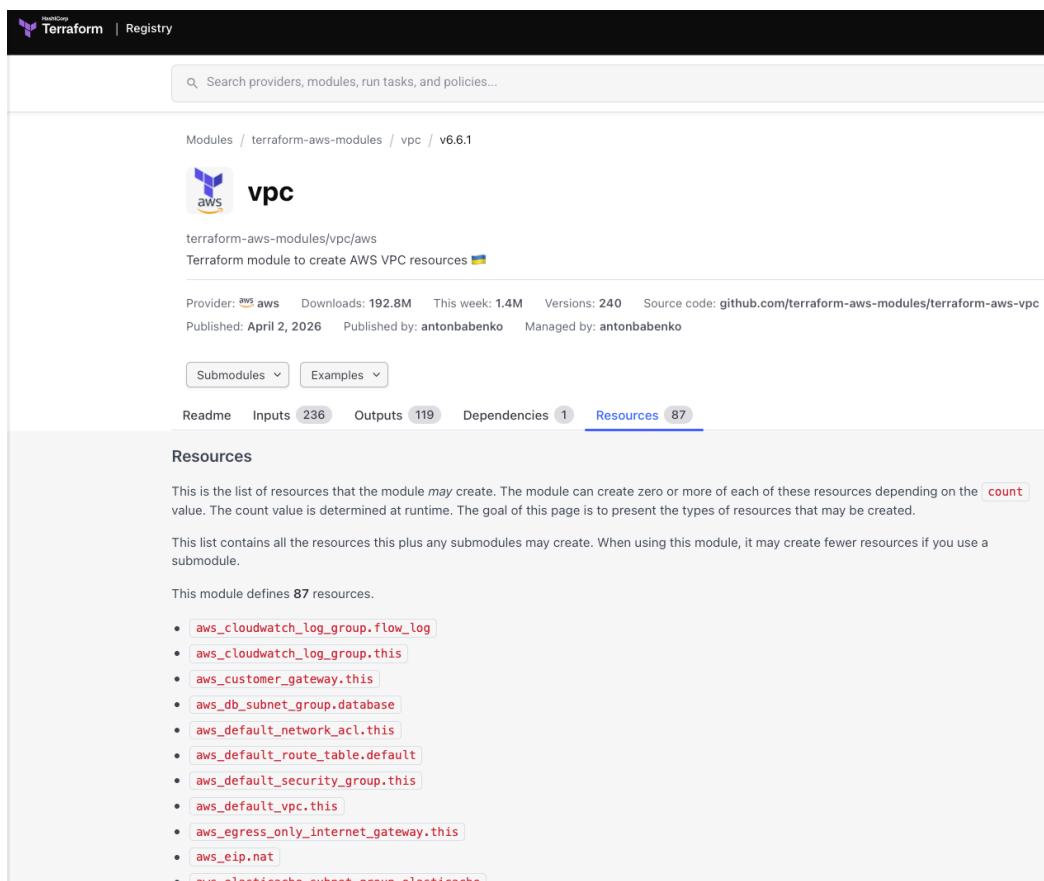
Examples

- AWS VPC module
- AWS IAM module
- GCP kubernetes module





If we check for example AWS VPC module, we can see the resources it creates:



The module will create everything for us, we need to provide the parameters. The parameters we can provide are define in the 'input' tab

The screenshot shows the Terraform Registry page for the `terraform-aws-modules/vpc/aws` module. The page header includes the Terraform logo and 'Registry' link. A search bar is at the top. The module name 'vpc' is prominently displayed with the AWS logo. Below it, the description reads 'Terraform module to create AWS VPC resources'. Metadata includes 'Provider: aws', 'Downloads: 192.8M', 'This week: 1.4M', 'Versions: 240', 'Source code: github.com/terraform-aws-modules/terraform-aws-vpc', 'Published: April 2, 2026', 'Published by: antonbabenko', and 'Managed by: antonbabenko'. Navigation tabs for 'Readme', 'Inputs (236)', 'Outputs (119)', 'Dependencies (1)', and 'Resources (87)' are shown. The 'Inputs' tab is active, displaying 'Optional Inputs' with descriptions and default values for `amazon_side_asn`, `azs`, `cidr`, and `create_database_internet_gateway_route`. A 'Module Downloads' section shows statistics for the latest version (6.6.1). 'Provision Instructions' are provided, including a code snippet for using the module in a Terraform configuration.

And we also can see what objects get created in this module if we go to the 'output' tab

This screenshot shows the same Terraform Registry page for the `terraform-aws-modules/vpc/aws` module, but with the 'Outputs' tab selected. The 'Outputs' tab displays a list of output objects created by the module, including `azs`, `cgw_arns`, `cgw_ids`, `database_internet_gateway_route_id`, `database_ipv6_egress_route_id`, `database_nat_gateway_route_ids`, and `database_network_acl_arn`, each with a brief description. The 'Module Downloads' and 'Provision Instructions' sections remain visible on the right side of the page.

And we can also see if the module depends on other modules and/or other providers-> 'dependency' tab/section

Modules / terraform-aws-modules / vpc / v6.6.1

vpc

terraform-aws-modules/vpc/aws

Terraform module to create AWS VPC resources

Provider: aws Downloads: 192.8M This week: 1.4M Versions: 240 Source code: github.com/terraform-aws-modules/terraform-aws-vpc

Published: April 2, 2026 Published by: antonbabenko Managed by: antonbabenko

Submodules Examples

Readme Inputs 236 Outputs 119 Dependencies 1 Resources 87

Dependencies

Dependencies are external modules that this module references. A module is considered external if it isn't within the same repository.

This module has **no external** module dependencies.

Provider Dependencies

Providers are Terraform plugins that will be automatically installed during **terraform init** if available on the Terraform Registry.

- aws (hashicorp/aws) >= 6.28

Module Downloads All versions

Downloads this week 1.4M

Downloads this month 4.3M

Downloads this year 32.1M

Downloads over all time 192.8M

Provision Instructions

VPC module doesn't have module dependencies 🙅

But it has one Provider dependency -> it needs to install the AWS provider like we did manually at the beginning of Module 12 - chapter 3

Module 12 - chapter 13
Saturday Found in attachments
DevOps_Nano — iCloud

Module 12 - chapter 11
Wednesday ...local Terraform Project The project fil...
DevOps_Nano — iCloud

Module 12 - chapter 3
Monday ...terraform init Q: I can have the depende...
DevOps_Nano — iCloud

Module 12 - chapter 2
14/06/2026 ...Install Terraform & Setup Terraform P...
DevOps_Nano — iCloud

...Terraform
12/06/2026 Found in attachments
DevOps_Nano — iCloud

Attachments

- cd terraform PNG Image - 215 KB
- 12 - Terraform Handout.pdf PDF Document - 8.3 MB
- N terraform init PNG Image - 132 KB
- Pasted Graphic 445.png PNG Image - 1.4 MB
- Pasted Graphic 446.png PNG Image - 1.3 MB

And I also can have a dedicated file to have the dependencies declarations -> **providers.tf** (I can name it however I want)

```

12 access_key = "AKIA2J6YSYB2DWTZUS47"
13 secret_key = "N1cXYQacpNgNdLWG+hMY0g5HKYcU9x1QP6wJ+m6"
14 }

```

terraform providers.tf

```

1 provider "aws" {
2   region = "eu-west-2"
3   access_key = "AKIA2J6YSYB2DWTZUS47"
4   secret_key = "N1cXYQacpNgNdLWG+hMY0g5HKYcU9x1QP6wJ+m6"
5 }

```

terraform

```

1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8 }

```

Now we can install the provider:
-> terraform init

```

nana@macbook: /Users/nana/terraform
% terraform init

```

terraform init

- initializes a working directory
- installs providers defined in the Terraform configuration

So whenever we use this module and we execute 'terraform init' all the providers that are defined inside the module will be automatically installed.

So when we use Modules we don't need to define the providers in the main.tf or the providers.tf, we just reference the module and with 'terraform init' the providers in the module get install.

— tangent —

Q:

is terraform provider a plugin?

A:

● Morning! Yes, a Terraform provider is a plugin. Terraform core doesn't know

anything about AWS, Azure, etc. — providers are separate binaries that Terraform downloads and communicates with via a plugin protocol (gRPC).

When you run `terraform init`, it downloads the provider plugins declared in your `required_providers` block into the `.terraform` directory.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

That `hashicorp/aws` is fetched from the Terraform Registry and acts as the plugin that knows how to talk to the AWS API.

— end tangent —

Another example is AWS EKS module, if we look at the dependency section

The screenshot displays the Terraform Registry page for the `eks` module. The page header shows the module name `eks` and the version `Version 21.23.0 (latest)`. The description states it is a "Terraform module to create Amazon Elastic Kubernetes (EKS) resources". The provider is `aws`, with 158.9M downloads and 316 versions. The `Dependencies` section lists external modules and provider dependencies. The `Module Downloads` sidebar shows statistics for the current version. The `Provision Instructions` box shows the module configuration code.

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "21.23.0"
}
```

EKS module will install 5 providers when executing `'terraform init'`

Also if we want to use this module, the way we reference it is using this code 📌 that is provided in the terraform website:

Provision Instructions

Copy and paste into your Terraform configuration, insert the variables, and run `terraform init`:

```
module "eks" {  
  source = "terraform-aws-  
modules/eks/aws"  
  version = "21.23.0"  
}
```

[Copy configuration](#) 